

[Marcar como hecha](#)

11.5. Atributos de petición, sesión y aplicación

En una típica tienda online, la aplicación principal que controla la tienda es un carro de la compra. Una tienda online le proporciona una interfaz navegable para un catálogo. Puede seleccionar artículos en el catálogo y añadirlos al carro de la compra. Una vez que ha añadido todos los artículos requeridos al carro, se dirige a la caja y presenta un pedido de los artículos en el carro.

Sin embargo, una aplicación de este tipo no es tan sencilla como parece. ¿Dónde se mantiene el carro de la compra? Puesto que el cliente es "infradotado", es responsabilidad del servidor mantener el carro no sólo de cara a nosotros sino para el resto de usuarios que pueden estar navegando el catálogo simultáneamente y añadiendo artículos a sus respectivos carros de la compra. Para que este mecanismo funcione, el servidor debe poder distinguir individualmente a cada usuario y, de acuerdo con ello, debe mantener cada carro ¿Cómo construimos esta función en una aplicación Web?

Éste es un requisito típico para aplicaciones Web, en las que la "actividad de usuario" tiene lugar en múltiples solicitudes y respuestas, y es necesario que el servidor asocie algún tipo de exclusividad a cada usuario. Analizaremos lo que se conoce como registro de sesión para enfrentarnos a este requisito. Examinaremos algunos enfoques de registro de sesión, en particular los utilizados por el API Servlet Java.

Además del registro de sesión, consideraremos otra instalación útil que proporciona el API Servlet, los servlets son parte de las aplicaciones Web y la noción de una aplicación Web se construye en lo que se denomina contexto de servlet. En términos sencillos, un contexto es una visión de la aplicación Web y el contenedor Web. Los servlets de una aplicación pueden utilizar este contexto para intercambiar información, o colaborar con otros servlets, acceder a recursos, registrar eventos, etc. Analizaremos los API Servlet relacionados con el contexto de servlet.

Aunque el análisis de este capítulo se centra en los servlets, las ideas generales también son aplicables a aplicaciones Web de base JSP.

Protocolo sin estado y sesiones

HTTP es un protocolo sin estado. Un cliente abre una conexión y solicita algún recurso o información. El servidor responde con el recurso solicitado (si está disponible), o envía un estado de error HTTP. Después de cerrar la conexión, el servidor no recuerda ninguna información sobre el cliente. Por lo tanto, el servidor considera la siguiente solicitud procedente del mismo cliente como una solicitud nueva, sin relación alguna con la solicitud anterior. Ésta es la función de un protocolo HTTP sin estado.

Un protocolo con estado es aquel en el que la respuesta a una solicitud dada puede depender no sólo de la solicitud actual sino también del resultado de solicitudes anteriores,

Normalmente, en un protocolo con estado, múltiples solicitudes y respuestas de cliente son enviadas a través de una única conexión de red entre el cliente y el servidor. Basándose en esta conexión, el servidor puede identificar tales solicitudes como parte integrante de una única sesión. Por ejemplo, considere el protocolo de transferencia de archivos (FTP). FTP es un protocolo con estado, con múltiples solicitudes y respuestas de cliente en una única conexión en una sesión determinada. En este caso, la conexión se establece con el primer comando OPEN y se cierra después del comando EXIT (a no ser, por supuesto, que un fallo de la red termine la conexión). Consecuentemente, el servidor FTP puede asociar todas las solicitudes de cliente a una única sesión con el cliente. El servidor también puede tomar decisiones basadas en el estado de la sesión. Por ejemplo, un servidor FTP puede limitar el número de solicitudes GET (para la descarga de archivos) dentro de una sesión.

Pero, ¿por qué es importante tener estado? Un protocolo con estado le ayuda a desarrollar una lógica compleja de aplicación en múltiples solicitudes y respuestas. Consideremos un banco online. Cuando solicita una página que contiene los balances de todas sus cuentas, el servidor debe poder verificar que es el verdadero titular de la cuenta y que ha demostrado sus credenciales al banco online. Sin embargo, cuando el protocolo no tiene estado, será necesario que envíe sus credenciales con cada solicitud; de hecho, cada transacción comercial deberá tener lugar en una única solicitud, Este método no es recomendable para largas transacciones comerciales que deben desarrollarse en múltiples solicitudes, como en el banco online, o en el carro de la compra que hemos visto hace un momento.

Para implementar transacciones comerciales flexibles en múltiples solicitudes y respuestas, necesitamos dos instalaciones:

?

- **Sesión** El servidor debe ser capaz de identificar que una serie de solicitudes de un único cliente forman una única "sesión" de trabajo. Asociando

una solicitud específica para que pertenezca a una sesión específica de trabajo, la aplicación del carro de la compra o el banco online pueden distinguir un usuario de otro,

- **Estado** El servidor debe ser capaz de recordar información relacionada con solicitudes anteriores y otras decisiones comerciales que son realizadas para las solicitudes. Es decir, la aplicación debería ser capaz de asociar estado con cada sesión. En el caso de la aplicación del carro de la compra, el posible estado podría incluir las categorías de artículos favoritos del usuario, perfil del usuario, o incluso el mismo carro de la compra.

Sin embargo, en el caso de HTTP, las conexiones son cerradas al final de cada solicitud y por ello los servidores HTTP no pueden utilizar la noción de conexiones para establecer una sesión. HTTP es un protocolo sin estado, que se ocupa de solicitudes y respuestas, que son transacciones sencillas y aisladas.

Es perfecto para navegar webs sencillas, donde cada solicitud produce normalmente descargas de contenido estático. El servidor no necesita saber si una serie de solicitudes procede del mismo o de diferentes clientes, o si esas solicitudes están relacionadas o son independientes. Pero éste no es el caso de las aplicaciones Web, donde necesitamos la capacidad para realizar transacciones comerciales en múltiples solicitudes y respuestas.

Con HTTP, en una transacción que produce múltiples solicitudes y respuestas, el servidor Web no puede determinar si todas las solicitudes proceden del mismo cliente. Un cliente, por lo tanto, no puede establecer un diálogo con el servidor Web para realizar una transacción económica. Sin embargo, el objetivo de HTTP ha sido proporcionar una recuperación de información rápida y ligera en Internet y un protocolo sin estado es lo más adecuado para estos requisitos.

Aparte de poder registra usuarios basándose en la noción de sesiones, el servidor también debería ser capaz de recordar cualquier información necesaria dentro de una sesión. Es decir, el programador de **la aplicación debe ser capaz de especificar qué datos deben ser recordados en una sesión**, de modo que la aplicación pueda utilizar esa información para tomar decisiones más informadas. Esto resulta muy útil transacciones o procesos que producen múltiples solicitudes y respuestas. Con protocolos con estado, el servidor asocia un estado a la conexión: recuerda quién es el usuario y qué está haciendo, con la a de la conexión. Sin embargo, esto no se puede conseguir con HTTP, puesto que la vida de una conexión se limita a una única solicitud.

Con el tiempo, diversas estrategias han evolucionado para tratar el registro de sesión y para gestionar el contenido en una sesión. El API Java Servlet proporciona funciones para registrar sesiones y mantener el estado en una sesión determinada. Con la ayuda de estas funciones, el servidor puede asociar todas las solicitudes juntas y saber que todas proceden del mismo usuario. También puede asociar un estado con la conexión: recuerda quién es el usuario y qué está haciendo.

Enfoques del registro de sesión

Existen fundamentalmente cuatro modos de enfocar el registro de sesión:

- Reescritura URL
- Campos de formulario ocultos
- Cookies
- Sesiones que utilizan la capa Secure Sockets Layer (SSL)

Observe que algunos libros sugieren la autenticación de usuario como un medio de registro de sesión. Pero la autenticación de usuario es una decisión de nivel de aplicación y no todas las aplicaciones requieren autenticación de usuario para todos los recursos de una aplicación Web.

Aunque los cuatro enfoques anteriores difieren en detalles de implementación, todos ellos están basados en un sencillo truco: intercambiar algún tipo de testigo entre el cliente y el servidor.

Considere un cliente C y un servidor S. Cuando C envía una solicitud a S por primera vez, S da a C un testigo único. Este testigo puede ser tan sencillo o tan complejo como queramos que sea: un número, Un nombre de usuario, o una cadena ID de sesión. Siempre que C visita S de nuevo, también envía el testigo junto con la solicitud. S puede ahora reconocer a C por su testigo.

Esta sencilla técnica para el registro de sesión puede ser adaptada de diferentes formas, basándose en cómo el testigo puede ser representado e intercambiado. Consideremos ahora los cuatro enfoques anteriores y cómo estos enfoques representan los testigos y cómo estos testigos son intercambiados en estos enfoques:

- **Reescritura de URL** En este enfoque, el testigo es integrado en cada URL. En cada página generada dinámicamente, el servidor integra un parámetro extra de consulta, o información de ruta extra, en cada URL de la Página. Cuando el cliente envía las solicitudes utilizando estos URL, el testigo es transmitido de nuevo al servidor. Este enfoque se denomina reescritura de URL, ya que conlleva describir URL en el contenido de la respuesta para integrar en testigo extra.
- **Campos de formulario ocultos** Este enfoque es similar a la reescritura de URL. En lugar de describir cada URL, el servidor integra campos ocultos en cada formulario. Cuando el cliente envía un formulario, los campos adicionales también serán enviados en la solicitud. El servidor puede utilizar estos parámetros para establecer y mantener una sesión.
- **Cookies** Los cookies fueron inventados por Netscape y son una de las formas más refinadas de testigo que clientes y servidores pueden intercambiar. A diferencia de describir URL o de utilizar campos de formulario ocultos, los cookies pueden ser intercambiados en cabeceras ' ? solicitudes y respuestas, y por lo tanto no suponen manipular la respuesta generada para que contenga un testigo.
- **Sesiones de Secure Socket Layer (SSL)** SSL es una tecnología de cifrado que se ejecuta sobre TCP/IP y bajo protocolos de nivel de aplicación como

HTTP. SSL es la tecnología utilizada en el protocolo HTTPS. Los servidores con SSL pueden autenticar a clientes habilitados con SSL y utilizar una conexión cifrada entre cliente y servidor. En el proceso de establecer una conexión cifrada, tanto el cliente como el servidor generan lo que se denomina "claves de sesión", que son claves simétricas utilizadas para cifrar y descifrar mensajes. Los servidores basados en el protocolo HTTPS pueden utilizar la clave simétrica del cliente para establecer una sesión.

Pero ¿cómo podemos elegir entre estos enfoques y establecer y mantener sesiones en nuestras aplicaciones Web? La respuesta es que, con servlets, es responsabilidad del contenedor Web proporcionar acciones básicas para crear y mantener sesiones. La especificación del servlet permite a los contenedores Web utilizar la reescritura de URL, cookies, o sesiones SSL para el registro de sesión. La verdadera técnica utilizada para establecer y registrar una determinada sesión depende de la capacidad del lado del cliente para participar en sesiones. El API Servlet también le ofrece una interfaz, cuyos objetos le permiten manipular el ciclo de vida de la sesión y asociar estado con sesiones. Existen ciertas directrices que certifican que el registro de sesión con independencia de la técnica utilizada.

Antes de examinar los componentes del API Servlet de registro de sesión, analicemos los tres primeros enfoques

Reescritura de URL

Normalmente, los navegadores generan solicitudes HTTP con pares valor clave en cadenas con ? e =, al enviar formularios con ACTINO tipo GET. Solicitudes POST en sus formularios, los dos parámetros de consulta formarán parte del cuerpo. Cada información extra de ruta del URL así como los parámetros de consulta pueden ser extraídos del objeto HttpServletRequest desde dentro de su servlet.

La reescritura de URL utiliza la misma técnica para integrar testigos únicos específicos del cliente en cada URL. Por ejemplo, a continuación, cada ruta de URL tiene una cadena uid=joe adjunta, donde uid es un parámetro exclusivo para cada usuario, con valor joe.

Por el momento, asumamos que el servlet ha generado este contenido dinámico, posponiendo la cuestión de cómo puede realizarse esto con el API Servlet. Cuando el servlet genera el fragmento de código anterior, integra en uid dentro de cada URL. Cuando el usuario hace clic en cualquiera de estos hipervínculos, el uid es pasado junto con la solicitud al contenedor Web. El contenedor Web puede entonces obtener el valor de la ruta desde la solicitud. Como hemos analizado en la sección anterior, el servidor envía el testigo como parte del URL y el cliente lo envía de vuelta junto con la solicitud, otorgando así al contenedor la capacidad de registrar la sesión de joe.

Este enfoque se denomina reescritura de URL ya que conlleva describir todos los URL para incluir un testigo exclusivo en la ruta de URL. Aunque hemos utilizado un parámetro uid en el ejemplo anterior, el nombre del parámetro especificado en la especificación del servlet es jsessionid. El verdadero URL generado es similar al siguiente:

<http://www.myserver.com/servlet/usrmenu;jsessionid=123456789>

La reescritura de URL requiere que todas las páginas de la aplicación sean generadas dinámicamente. La reescritura de URL no puede ser aplicada en páginas HTML porque el exclusivo parámetro de ruta URL (el session) es dinámico y difiere entre usuarios. Observe también que el jsessionid (o el uid del ejemplo anterior) es un parámetro de ruta y no un parámetro de consulta (como las solicitudes GET). Los parámetros de consulta son pares valor- nombre separados por "%", mientras que el parámetro de ruta jsessionid=<..> es una ruta de recurso codificada.

Campos de formulario ocultos

En este enfoque, el testigo exclusivo está integrado en cada formulario HTML. Por ejemplo, el siguiente HTML especifica un control de entrada de tipo Hidden:

```
<input type="hidden" name="uid" value="joe">
```

Cuando la solicitud es enviada, el servidor recibe el testigo como parte de la solicitud. Observe que, similar a la reescritura de URL, el contenido anterior debe ser generado dinámicamente integrando el parámetro oculto. Además, cada solicitud debe incluir una presentación de formulario y por ello puede no ser aplicable a todos los tipos de páginas.

La especificación de servlet no utiliza este enfoque.

El tercer recurso: **Cookies**, lo veremos en la siguiente página

Última modificación: martes, 14 de marzo de 2023, 10:27